

hyper2kvm - Enterprise VM Migration Toolkit

hyper2kvm

Enterprise-Grade VM Migration Toolkit

Any Hypervisor → KVM with Zero-Downtime & Automated Fixes

Author: Susant Sahani License: LGPL-3.0 | Python 3.10+ | PyPI: hyper2kvm

The Problem

Migrating VMs between hypervisors is painful:

- **Disk format incompatibility** — VMDK, VHDX, VHD, OVA all need conversion
- **Boot failures** — wrong drivers, broken GRUB, stale fstab entries
- **Manual intervention** — trial-and-error fixing after each migration
- **No Windows support** — VirtIO drivers must be injected manually
- **No automation** — each VM is a one-off effort

Traditional tools convert the disk and say “*boot and hope.*”

The Solution: hyper2kvm

Deterministic offline fixes ensure **first-boot success**.

Traditional Tools	hyper2kvm
Convert disk format only	Convert + deep inspection + repair
“Boot and hope”	Deterministic first-boot success
Manual fixes required	Automated bootloader, fstab, driver fixes
Trial and error	Validated pipeline with smoke tests

At a Glance

Metric	Value
VMCraft APIs	480+
Input Formats	8 (VMDK, OVA, OVF, VHD, VHDX, AMI, Azure VHD, Raw)
Supported OS Versions	35+ (Linux & Windows)
LVM Activation Time	0.71s (7x faster)
Test Suite	380+ tests
Performance vs Traditional	5-7x faster
Host Protection	100% (sandboxed LVM)
Dependencies	Pure Python (zero C libs)

Architecture Overview

Source VM	hyper2kvm Pipeline	Target
+-----+	+-----+	+-----+
VMware	1. Disk Conversion (qemu-img)	KVM
Hyper-V	2. VMCraft Inspection (OS, drivers, boot)	libvirt
Azure	3. Offline Fixes (GRUB, fstab, initramfs)	KubeVirt
Raw/OVA	4. Driver Injection (VirtIO)	k3s/OCF
+-----++	5. Validation & Smoke Test	+-----+
	+-----+	

VMCraft Engine

The core of hyper2kvm — a **pure Python** VM manipulation engine.

480+ API methods for comprehensive VM inspection and modification.

Performance Comparison

Metric	Traditional (libguestfs)	VMCraft
Launch Time	9.7s	1.9s
Memory Usage	280 MB	95 MB
System Calls	14,200	8,500

5-7x faster launch, **66% less** memory, **40% fewer** system calls.

Quick Example

```
from hyper2kvm.core.vmcraft import VMCraft

with VMCraft() as g:
    g.add_disk("/vms/server.qcow2")
    g.launch() # ~1.9 seconds

    os_info = g.inspect_os()
    print(f"Detected: {os_info.product_name}")

    g.write("/etc/motd", "Migrated to KVM!\n")
```

Key Features

Automated Offline Fixes

- **Bootloader repair** — GRUB/GRUB2 configuration for VirtIO disks
- **fstab stabilization** — Convert device names to UUID/PARTUUID/LABEL
- **initramfs regeneration** — Inject VirtIO drivers into boot image
- **Network configuration** — Adapt interfaces for KVM environment

Windows Migration

- **VirtIO driver injection** — storage, network, balloon, serial
- **Registry modification** — boot-critical driver registration
- **License preservation** — OEM, Retail, MAK, KMS detection
- **Active Directory** — domain membership detection & rejoin
- **Supported:** Windows 7–11, Server 2008 R2–2025

Live Fix (Zero Downtime)

- SSH-based remote fixes on running VMs
- No shutdown required
- fstab stabilization, initramfs regen, driver fixes

Supported Platforms

Input Formats

VMDK | OVA | OVF | VHD | VHDX | AMI | Azure VHD | Raw

Linux Distributions

RHEL/CentOS 7–9 | Fedora | Ubuntu 18.04–24.04 | Debian 10–12 | SUSE/openSUSE | Photon OS | Flatcar | CoreOS | Amazon Linux | Oracle Linux | Rocky/Alma Linux

Windows Versions

Windows 7–11 | Server 2008 R2–2025 | x64 and x86

Deployment Targets

libvirt/KVM | KubeVirt | Kubernetes | OpenShift | k3s

Migration Workflow

Step 1: Define Configuration (YAML)

```
cmd: local
vmdk: /path/to/server.vmdk
output_dir: /output
to_output: server.qcow2
out_format: qcow2
fstab_node: stabilize-all
regen_initramfs: true
no_grub: false
libvirt_test: true
```

Step 2: Run Migration

```
sudo h2kvmctl --config migration.yaml
```

Step 3: Deploy

```
sudo virsh define server.xml
sudo virsh start server
```

Kubernetes & OpenShift Integration

One-Command Deploy to Kubernetes

```
sudo h2kvmctl --config migration.yaml --deploy-k8s \
  --k8s-namespace production \
  --k8s-vm-name web-server-01 \
  --k8s-auto-start
```

Automated pipeline: 1. VMDK to QCOW2 conversion with offline fixes 2. Upload to Kubernetes PVC 3. Create KubeVirt VirtualMachine resource 4. Start VM and wait for ready

OpenShift Support

- OperatorHub integration
 - SecurityContextConstraints
 - Routes & OAuth
 - OLM bundle packaging
-

Enterprise Features

Security

- **LUKS/TPM2 encryption** support
- **Host VG protection** — 100% sandboxed LVM activation
- **Secure boot** compatibility

Batch Processing

- Parallel migration of multiple VMs
- Progress tracking and reporting
- Manifest-based orchestration

Daemon Mode

- Background processing via systemd service
- REST API for integration
- Metrics and monitoring

Observability

- TUI dashboard (Textual / Curses / CLI fallback)
- Rich progress bars with ETA
- Debug bundle collection

LVM Enterprise Safety

The Risk

Activating LVM inside a migrated disk image can accidentally activate **host** volume groups — potentially corrupting the host OS.

hyper2kvm's Solution

- **Sandboxed activation** — only target VG is activated
 - **Host VG exclusion** — automatic detection and protection
 - **7x faster** — 0.71s activation (down from ~5s)
 - **Tested** on RHEL 8.8 & openSUSE 15.4
-

Project Structure

```
hyper2kvm/  
  cli/           # CLI interface (Click-based)  
  core/          # Core engine, VMCraft, progress  
  converters/    # Disk format converters (qemu-img)  
  fixers/        # Offline fix modules  
    bootloader/  # GRUB repair  
    windows/    # VirtIO injection, registry  
    live/        # SSH-based live fixes  
    offline/    # Mounted-disk operations  
  providers/    # Source hypervisor integrations  
    vmware/     # vSphere, ESXi  
    azure/      # Azure VHD  
  orchestration/ # Batch, manifest, pipeline  
  infrastructure/ # K8s deployers, SSH  
  vmcraft/      # VM manipulation engine  
  systemd/     # systemd integration tools  
  luks/        # Encryption support
```

Getting Started

Installation

```
pip install "hyper2kvm[full]"
```

System Dependencies (Optional)

```
# Fedora/RHEL
sudo dnf install -y qemu-img qemu-system-x86

# Windows migration support
sudo dnf install -y ntfs-3g libhivex-bin
```

CLI Commands

```
h2kvmctl --version      # Interactive CLI
hyper2kvm --version     # Daemon / systemd mode
```

Both commands are functionally identical.

Summary

hyper2kvm is a production-ready, enterprise-grade toolkit that eliminates the pain of VM migration:

- **Deterministic** — no more “boot and hope”
- **Comprehensive** — 35+ OS versions, 8 input formats
- **Fast** — 5-7x faster than traditional tools
- **Safe** — 100% host protection, sandboxed operations
- **Automated** — one command from source to running VM
- **Cloud-native** — Kubernetes, OpenShift, KubeVirt ready
- **Pure Python** — zero C dependencies, runs anywhere

Links

- **PyPI:** <https://pypi.org/project/hyper2kvm/>
 - **GitHub:** <https://github.com/ssahani/hyper2kvm>
 - **License:** LGPL-3.0
-

hyper2kvm — Migrate Once, Boot Right.